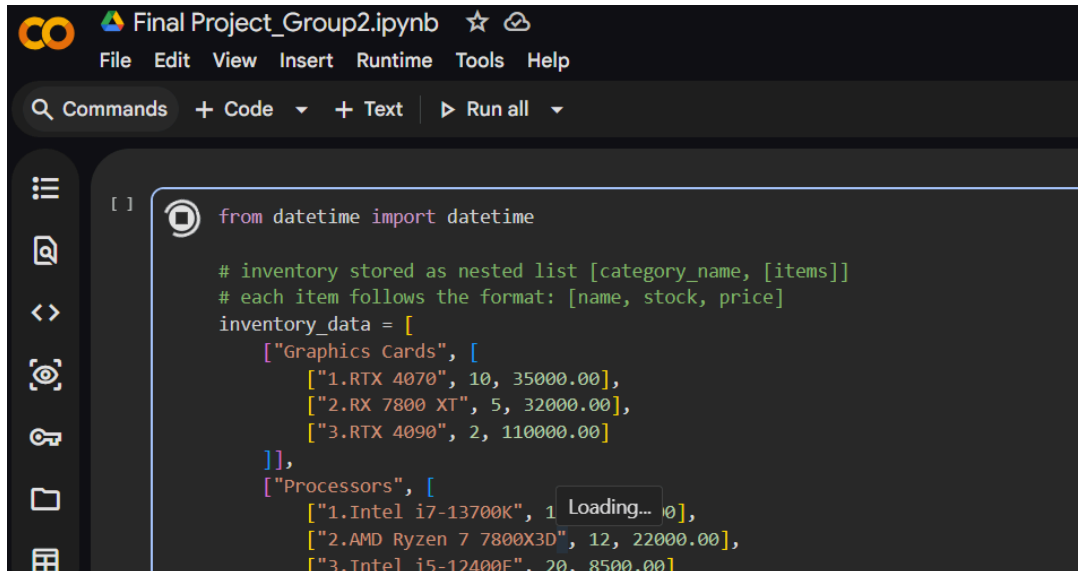


Final Examination	
Inventory Management System	
Course Code: CPE 009A	Program: Computer Engineering
Course Title: Object Oriented Programming 1	Date Performed: April 10, 2026
Section: CPE12S2	Date Submitted: April 24, 2026
Name(s): Group 2: CAMPO, RAMON BRENDT CANOY, HAIL CORRO, YAZINEL DE LEON, BENJ FEDERIC DELA PROVIDENCIA, DANE MATTHEW	Instructor: Engr. Maria Rizzete H. Sayo
1. Objectives The purpose of this program is to provide a simple, organized, and efficient way to manage a computer hardware inventory using a command line. It allows the users to view the availability of products, to update the stocks, to process the purchase of customers, and the records of all transactions.	
2. Intended Learning Outcome - To apply Object-Oriented Programming (OOP) concepts and data structures (lists, dictionaries, nested loops) in a real-world scenario. - To design a functional Command Line Interface (CLI) that handles user input, performs data validation, and manages stateful information like inventory and sales logs. - To demonstrate collaborative software development and documentation within a team environment	
3. Discussion The program is a Python-based Inventory Management System designed for retail environments, specifically computer hardware. It utilizes a structured list called <code>inventory_data</code> to store product categories (Graphics Cards, Processors, etc.), quantities, and pricing. Key functionalities include: - Inventory Tracking: Real-time monitoring of stock levels. - Transaction Processing: A sales module that generates official receipts, calculates totals, and automatically deducts stock upon purchase. - Restocking: A maintenance feature to add units to existing products (limited to 10 units per transaction to ensure controlled entry). - Sales Logging: A history feature that uses a <code>sales_log</code> to archive all past transactions for auditing.	
4. Materials and Equipments Hardware: Laptop and Phone for coding and communication. Software/IDE: Google Colab and Programiz Python Online Compiler. Collaboration Tools: Discord and Messenger for group meetings and file sharing. Storage: Google Drive for documentation and notebook storage.	
5. Procedures	

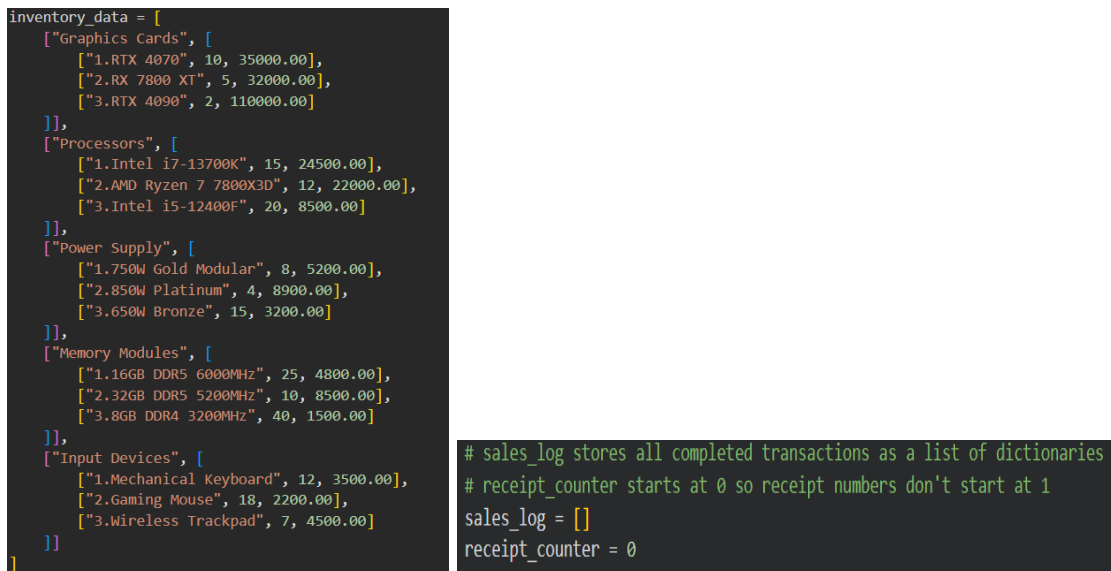
1. Environment Setup: Open Google Colab or Programiz and initialize the required Python libraries (e.g., `datetime`).



```
from datetime import datetime

# inventory stored as nested list [category_name, [items]]
# each item follows the format: [name, stock, price]
inventory_data = [
    ["Graphics Cards", [
        ["1.RTX 4070", 10, 35000.00],
        ["2.RX 7800 XT", 5, 32000.00],
        ["3.RTX 4090", 2, 110000.00]
    ]],
    ["Processors", [
        ["1.Intel i7-13700K", 1, 24500.00],
        ["2.AMD Ryzen 7 7800X3D", 12, 22000.00],
        ["3.Intel i5-12400F", 20, 8500.00]
    ]]
```

2. Data Structure Initialization: Define the `inventory_data` list and `sales_log` to store system information.



```
inventory_data = [
    ["Graphics Cards", [
        ["1.RTX 4070", 10, 35000.00],
        ["2.RX 7800 XT", 5, 32000.00],
        ["3.RTX 4090", 2, 110000.00]
    ]],
    ["Processors", [
        ["1.Intel i7-13700K", 15, 24500.00],
        ["2.AMD Ryzen 7 7800X3D", 12, 22000.00],
        ["3.Intel i5-12400F", 20, 8500.00]
    ]],
    ["Power Supply", [
        ["1.750W Gold Modular", 8, 5200.00],
        ["2.850W Platinum", 4, 8900.00],
        ["3.650W Bronze", 15, 3200.00]
    ]],
    ["Memory Modules", [
        ["1.16GB DDR5 6000MHz", 25, 4800.00],
        ["2.32GB DDR5 5200MHz", 10, 8500.00],
        ["3.8GB DDR4 3200MHz", 40, 1500.00]
    ]],
    ["Input Devices", [
        ["1.Mechanical Keyboard", 12, 3500.00],
        ["2.Gaming Mouse", 18, 2200.00],
        ["3.Wireless Trackpad", 7, 4500.00]
    ]]
```

```
# sales_log stores all completed transactions as a list of dictionaries
# receipt_counter starts at 0 so receipt numbers don't start at 1
sales_log = []
receipt_counter = 0
```

3. Core Function Definition: Program functions for displaying menus (`display_category_menu`) and showing detailed inventory lists (`show_inventory_details`).

```

# prints just the category names with numbers so the user can pick one
def display_category_menu():
    print("\nPlease select a category:")
    for i, item in enumerate(inventory_data, start=1):
        print(f" {i}. {item[0]}")

# shows the full details (model, stock, price) of items inside a category
def show_inventory_details(cat_idx):
    category = inventory_data[cat_idx]
    print(f"\n- - - {category[0].upper()} - - -")
    print(f"{'Model':<25} {'Stock':<10} {'Price':<10}")
    print("-" * 50)
    for model in category[1]:
        name, stock, price = model
        print(f"{'name':<25} {'stock':<10} {price:,.2f}")

```

4. Main Loop Implementation: Create a `while True` loop to keep the program running until the "Exit" option is chosen.

```

# keeps running until the user chooses to exit
while True:
    print("\n" + "=" * 45)
    print("      INVENTORY MANAGEMENT SYSTEM")
    print("=" * 45)
    print("  1. View Inventory")
    print("  2. Process Sale")
    print("  3. Restock Inventory")
    print("  4. Sales History")
    print("  5. Exit")

    choice = input("\nPlease enter your selection (1-5): ")

```

5. Menu Logic: Implement `match-case` or `if-elif` blocks to handle the five main features: View, Sale, Restock, History, and Exit.

```

# using match-case here instead of if-elif chain, cleaner for menu handling
match choice:
    case '1':
        print("\n" + "-" * 30)
        print("      Inventory")
        print("-" * 30)

        display_category_menu()

        sub_choice = input("\nPlease select a category (1-5): ")

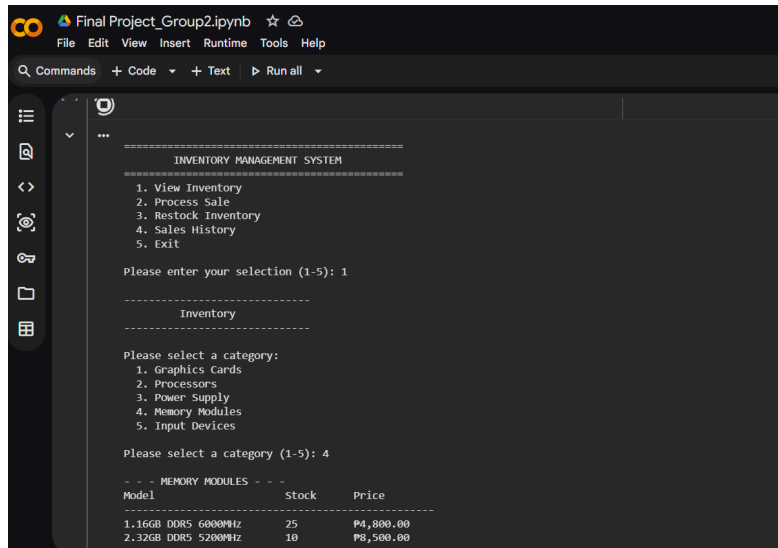
        if sub_choice.isdigit() and 1 <= int(sub_choice) <= len(inventory_data):
            show_inventory_details(int(sub_choice) - 1)
        else:
            print("Invalid selection.")

        return_to_menu()

    case '2':
        print("\n" + "-" * 31)

```

6. Testing & Validation: Run the program to ensure inputs (like non-numeric characters or excessive quantities) are handled without crashing.



```
=====
INVENTORY MANAGEMENT SYSTEM
=====
1. View Inventory
2. Process Sale
3. Restock Inventory
4. Sales History
5. Exit

Please enter your selection (1-5): 1

-----
Inventory
-----

Please select a category:
1. Graphics Cards
2. Processors
3. Power Supply
4. Memory Modules
5. Input Devices

Please select a category (1-5): 4

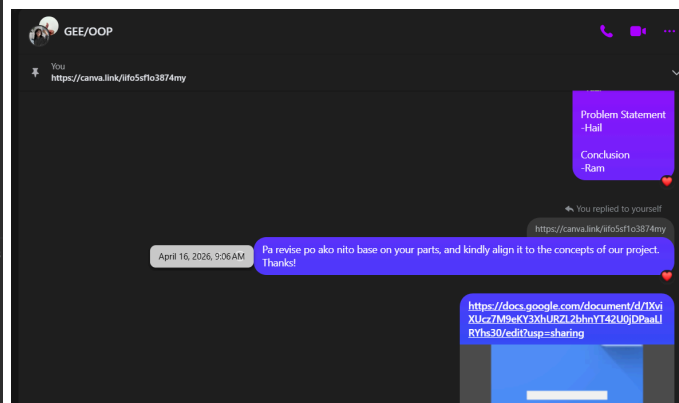
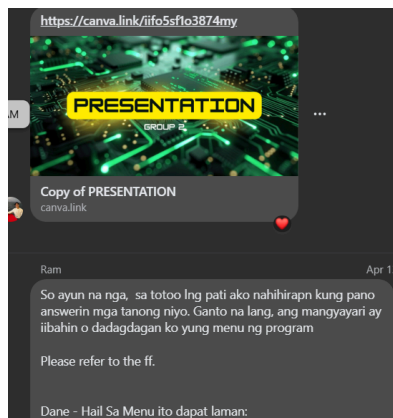
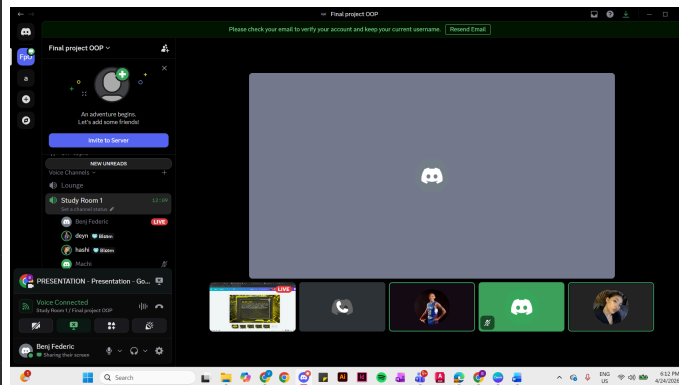
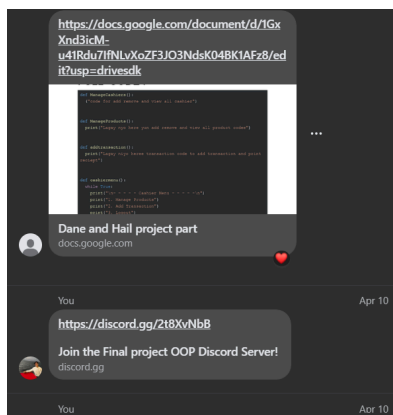
- - - MEMORY MODULES - - -
Model          Stock    Price
-----
1.16GB DDR5 6000MHz    25    P4,800.00
2.32GB DDR5 5200MHz    10    P8,500.00
3.16GB DDR5 5200MHz    10    P4,500.00
```

6. Output

Final Project_Group2.ipynb

7. Supplementary Activity

The Group utilized a dedicated **Discord Server ("Final project OOP")** and **Facebook Messenger** to coordinate tasks. Screenshots of the Discord environment show active collaboration in the "Study Room", and where code snippets were debugged and project milestones were discussed.



8. Conclusion

This program can help lots of people that's working in retail and other people starting a business which is not limited to computer shops only but it can also be used by other stores that has the same purpose, it also display how the code operates and what is the purpose of each line of code and whats their functions, which can help with other programers and students that can use this as their reference.

9. Assessment Rubric